

A draft model for a robot diffusion policy, revisited

Aarav Raina · September 2026 · second version of the May 2026 hackathon writeup

Abstract

In May I reported that pairing the `lerobot/diffusion_pusht` robot policy with a tiny draft model, the way LLM servers use speculative decoding, made it about 1.6x faster at a 10 Hz control rate. The hackathon README said 2.02x. This version re-runs the idea with the evaluation it needed. The draft is scored on episodes it never trained on, every strategy is scored on the same frames, the serving loop runs plan by plan, gates pay for every verifier call they make, and every strategy also drives the PushT simulator. The original rule gives no speedup, because it needs the verifier's action to decide whether to skip the verifier. Gates that decide without the verifier do skip calls, but they more than quadruple the action error on the way to 2x. Running the verifier with 10 or 5 denoising steps instead of 20 cuts model time 2x to 3.9x with no draft and stays within noise of the 20-step sampler in open loop. In the simulator the few-step samplers succeed about as often as the 20-step reference, within the noise of 50 rollouts, the draft never succeeds, and the original rule cuts success from 48% to 22% while saving nothing. The first version was a model cascade. The part of speculative decoding that carries over to diffusion policies works inside the denoising chain, and it already exists.

1. What the first version claimed

The May version paired a 4,482-parameter MLP draft (pusher position in, action out) with `lerobot/diffusion_pusht` as the verifier. At each decision it served the draft's action when it landed within `tau` pixels of the verifier's action. The writeup reported a 1.6x projected speedup at a 10 Hz deadline, bootstrap CI 1.30x to 2.12x, with 35% of decisions going to the draft and action error up 6%. The hackathon README reported 2.02x at 51.5% acceptance from an earlier run. I called it speculative decoding. It was a model cascade: a small model answers, and a large one takes over when a test fails (Kim et al. 2023 study the same pattern for language models).

Five things were wrong with it. The first one alone erases the speedup.

1. **The accept test needs the verifier.** Checking `||a_draft - a_verifier|| < tau` means running the verifier at every decision, but the cost model charged nothing for it on accepted frames. The synchronous loop in the first version measured 1.05x, which is 1x within timing noise, since the rule calls the verifier every time.
2. **Test frames were training frames.** The draft trained on episodes 0 to 49, and I evaluated it on episodes 0, 1 and 2. The verifier checkpoint was trained on every pusht episode.
3. **Quality and cost came from different serving modes.** I scored rejected frames on a fresh plan made at that exact frame and charged them one eighth of a call, as if eight frames shared the plan.
4. **The ablation compared different frames.** The fixed-sampler rows used 8 early frames of episode 0, and the speculative row used a 3-episode pool. On matched frames speculative was worse than plain

diffusion, 170 vs 160.

5. **The deadline set the speedup.** `tau` was the smallest value that got projected latency under 100 ms on the evaluation frames. Under that cost model speedup is about $1/(1 - \text{acceptance})$, so the reported number came out as baseline latency over 100 ms.

Smaller problems: the pool had 68 frames where I reported 72, because episode 1 is only 118 frames long and a guard skipped the rest silently. The verifier saw its current frame twice instead of the previous and current frame. The few-step baseline only tried DDPM with skipped steps. Every number came from one diffusion sample per frame.

2. Setup

Policy and data. `lerobot/diffusion_pusht` is a Diffusion Policy checkpoint (Chi et al. 2023) with 262.7M parameters. It was trained with a 100-step DDPM (denoising diffusion probabilistic model, Ho et al. 2020) schedule, sees 2 frames of history, plans 16 steps and executes 8 actions per plan. `lerobot/pusht` holds 206 human demonstrations of pushing a T-shaped block into place, 25,650 frames at 10 Hz.

Splits. Episodes 0 to 149 train the draft. Episodes 150 to 169 pick thresholds. Episodes 170 to 205 (4,808 frames) produce every open-loop number below. The verifier checkpoint was trained on all 206 episodes, so only the draft is held out. The closed-loop runs aren't scored against the demonstrations.

Draft. The same MLP and training recipe as the first version, trained on the train split. I train 5 copies with different seeds. The first one is served, and their disagreement feeds one of the gates.

Verifier samplers. The reference is DDPM with 20 steps, because that's what the first version used. The checkpoint's own default is 100 steps, five times the compute, and I didn't run it here. I also run DDPM with 10 and 5 steps, DDIM (Song et al. 2021) with 10 and 5 steps, and DDPM-20 executing 12 or 15 actions per plan instead of 8. All samplers use diffusers' default timestep spacing.

Serving loop. At each plan boundary a strategy either calls the verifier and executes its plan, or serves the draft for the next 8 frames. The verifier always sees the real previous and current frame. Every frame of every test episode is served and scored.

Gates. Each gate decides at a plan boundary and pays for every verifier call it makes. The harness hands gates the verifier's plans through a wrapper that raises if a gate reads a plan it didn't pay for or looks ahead.

- *Notebook rule.* The first version's rule. It calls the verifier at every boundary and serves the draft when the two agree within `tau`. It can never skip a call.
- *Previous agreement.* After a verifier call whose first action landed within `tau` of the draft, serve the draft for the next plan without calling, then check again. It can at most halve the calls.
- *Draft ensemble.* Serve the draft when the 5 draft copies agree with each other (spread below `tau`). It never needs the verifier to decide.

Thresholds come from quantiles of each gate's signal on the validation episodes. The reported point is the one with the fewest verifier calls whose validation error stays within 5% of the reference. The full curve on test is in Figure 2. No setting was chosen from test results.

Metrics. Open loop: squared error in pixels between the served action and the demonstrator's action, averaged over the x and y coordinates, over frames, and over 4 diffusion samples per boundary. This is the first version's metric, and section 3.1 shows where it misleads. Closed loop: every strategy drives the PushT simulator for up to 300 steps from the same 50 starting positions. I report the success rate (block coverage above 95%) and the best coverage reached.

Timing. A separate benchmark times one call of each sampler, round-robin over real inputs, 50 times each, on an Apple M-series laptop GPU through PyTorch's MPS backend. Time per frame is verifier calls per frame times the median call, plus the draft. That average is a throughput number. A loop that waits for each call stalls for the whole call at every boundary, so real-time control needs a loop that plans the next chunk while executing the current one. I check whether that could work but don't simulate it.

Statistics. Open-loop intervals are BCa bootstraps over whole test episodes, 10,000 resamples. Comparisons with the reference are paired by episode, on the same frames, with the same starting noise wherever plan boundaries line up. Non-inferiority uses a 5% margin, the same budget the gates had. Closed-loop success rates get Wilson intervals.

3. Results

3.1 Open loop

Table 1 has every verifier sampler and the draft on the 36 test episodes, with 4 diffusion samples at every plan boundary. The verifier trained on these episodes, so for the verifier this measures agreement on data it has seen.

strategy	MSE [95% CI]	vs reference [95% CI]	95% upper bound	variance across samples	ms / frame	speedup	pipelined 10 Hz
DDPM-20 (reference)	54.1 [41.2, 77.4]			7.1	140.1	1.00x	no
DDPM-10	51.2 [38.9, 72.3]	-2.9 [-7.5, +1.2]	+0.6	5.4	70.5	1.99x	yes
DDPM-5	46.4 [37.2, 59.6]	-7.7 [-19.3, -1.4]	-2.3	5.1	35.7	3.92x	yes
DDIM-10	49.7 [39.4, 63.3]	-4.4 [-20.1, +3.1]	+2.1	8.0	70.6	1.99x	yes
DDIM-5	49.8 [39.4, 64.1]	-4.3 [-19.0, +3.0]	+2.0	6.9	35.7	3.93x	yes
DDPM-20, chunk 12	70.6 [51.8, 105.2]	+16.6 [+10.1, +29.3]	+26.7	8.1	94.4	1.48x	no
DDPM-20, chunk 15	82.5 [58.7, 135.7]	+28.4 [+16.1, +58.6]	+52.1	9.9	76.9	1.82x	no

strategy	MSE [95% CI]	vs reference [95% CI]	95% upper bound	variance across samples	ms / frame	speedup	pipelined 10 Hz
draft only	382.1 [349.9, 419.5]	+328.1 [+292.2, +363.9]	+358.7	0.0	0.1		

Table 1. Open loop on episodes 170 to 205, whole episodes. MSE is in px^2 . "vs reference" is paired by episode. The upper bound is the one-sided 95% limit on that difference, against a non-inferiority margin of 2.7 (5% of the reference). "Variance across samples" is the part of the error that comes from the 4 diffusion samples disagreeing with each other; the rest is the error of their average. Time per frame uses the median benchmarked call: 1,090 ms for 20 steps, 548 ms for 10 and 278 ms for 5. "Pipelined 10 Hz" means even the slowest benchmarked call is short enough to plan the next chunk while the current one executes. A plan covers 15 frames, so an 8-action plan leaves 700 ms, a 12-action plan 300 ms and a 15-action plan none.

Most of this error sits in one place. For the reference, frames 0 to 7 of each episode average 692 and every later frame averages 13.4, so the first plan is 6% of the frames and 77% of the error. Every sampler starts each episode from a single repeated frame, with no motion history, while the demonstrator is moving fastest. Taken over whole episodes, all four few-step samplers are non-inferior to DDPM-20, but that result is mostly about the first plan.

Table 2 repeats the comparison from frame 16 on, where every sampler is past its first plan. I added this cut after a reviewer pointed out the first-plan effect, so treat it as a sensitivity analysis.

strategy	MSE from frame 16	vs reference [95% CI]	95% upper bound	margin	worse on
DDPM-20 (reference)	13.46				
DDPM-10	12.85	-0.61 [-2.05, +0.13]	+0.03	0.67	12 of 36
DDPM-5	13.36	-0.10 [-1.18, +0.93]	+0.76	0.67	16 of 36
DDIM-10	13.53	+0.07 [-2.42, +1.30]	+1.14	0.67	27 of 36
DDIM-5	14.02	+0.55 [-1.99, +1.94]	+1.76	0.67	27 of 36
DDPM-20, chunk 12	13.70	+0.24 [-2.95, +2.42]	+2.12	0.67	20 of 36
DDPM-20, chunk 15	17.13	+3.67 [+1.57, +8.16]	+7.22	0.67	26 of 36
draft only	337.09	+323.63 [+291.55, +360.68]			36 of 36

Table 2. The same test episodes from frame 16 on (4,232 frames). "Worse on" counts episodes where the strategy's error is above the reference's.

Past the first plan, DDPM-10 is non-inferior. DDPM-5 is within noise but misses the 5% margin narrowly. DDIM is within noise on average, but it's slightly worse on 27 of 36 episodes and can't be shown non-inferior. So fewer steps cost at most a few percent of this error, and open loop can't separate the few-step samplers from each other. Executing 12 actions per plan matches the reference past the first plan, and 15 actions adds about 27%.

The metric also favors samplers that vary less. Squared error against one demonstrator splits exactly into the error of the average sample plus the variance between samples, so less variance scores better by construction. Fewer DDPM steps lower that variance, from 7.1 to 5.4 at 10 steps and 5.1 at 5 steps, and both drops are significant. Keeping several ways to act is what diffusion policies are for, so the closed-loop runs in section 3.3 decide whether fewer steps is safe.

A pipelined loop would also cost accuracy that these numbers don't show. It executes actions further into each plan, because the plan came from an older observation. In the cached 15-action DDPM-20 plans, from frame 16 on, actions 0 to 7 of a plan average 13.2, actions 3 to 10 average 16.6 (+26%) and actions 4 to 11 average 17.6 (+33%).

The draft's error is 382 on test and 442 on validation, about seven times the verifier's. From frame 16 on it's 337, about 25 times.

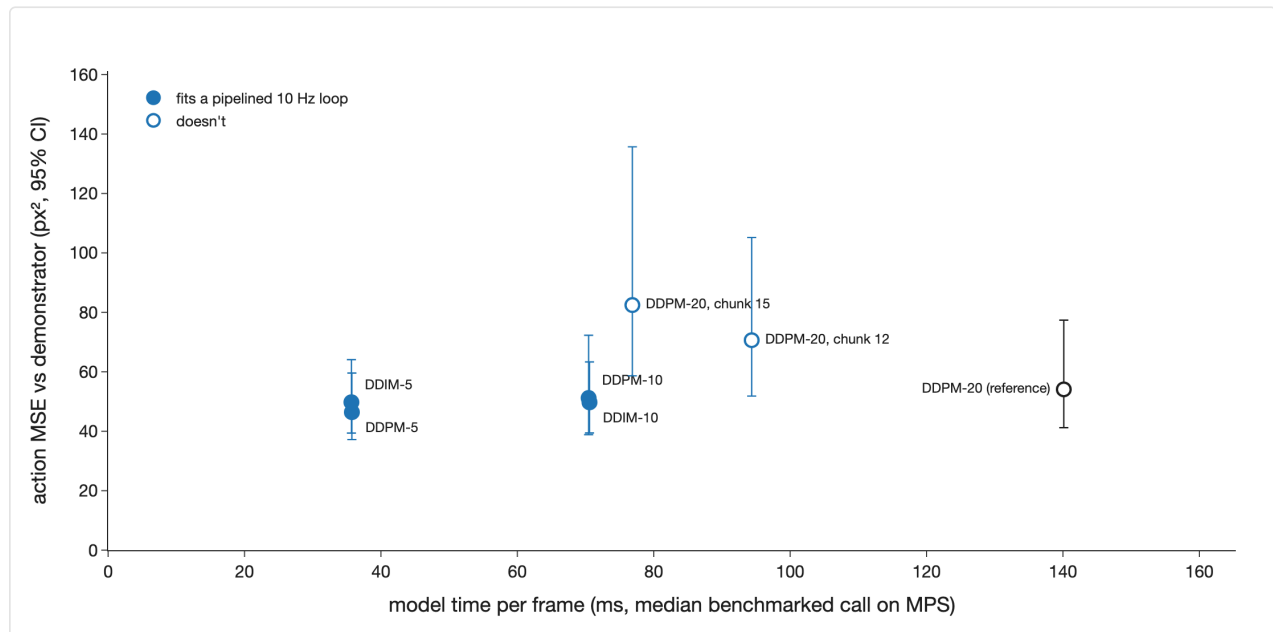


Figure 1. Open-loop error over whole episodes against model time per frame for every verifier sampler. Filled points could keep up with 10 Hz in a pipelined loop. Error bars are 95% intervals over test episodes.

3.2 Gates

No gate found a threshold that kept validation error within 5% of the reference, so at their chosen thresholds all three fall back to calling the verifier at every boundary. The ensemble gate came closest. Its lowest threshold missed the budget by 0.05 and would have saved 1.3% of calls. Figure 2 shows what each gate trades across its whole threshold range on test.

The notebook rule never saves anything. It pays for the verifier at every boundary at every threshold, so all it does is swap verifier plans for draft plans and add error, from +3.7 when 1.2% of plans go to the draft up to +274 at 95%.

The two gates that decide without the verifier do save calls, at a steep price. At their smallest thresholds they save 1% to 2% of the compute for +4.5 and +2.5. Near 1.1x they add +39.6 and +31.6. The previous-agreement gate peaks at 1.93x with +186, since it has to check the verifier at least every other plan. The ensemble gate reaches 1.98x at +181. The small penalties at the lowest thresholds are pointwise intervals among 42 curve points, so treat them as suggestive. The large ones hold under any correction. Gates and fewer steps could be combined, but I only tested gates in front of DDPM-20.

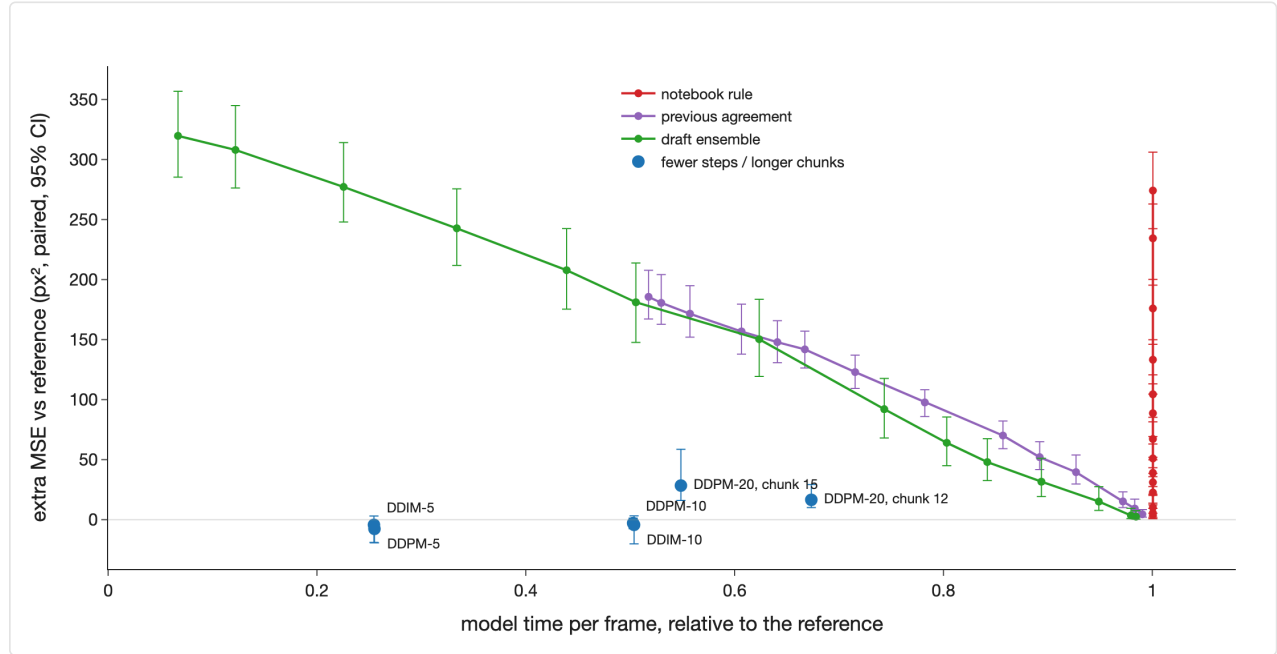


Figure 2. Extra error over whole episodes against compute, both relative to the reference and paired on the same test frames. Lines are the three gates across thresholds picked from validation quantiles. Blue points are the verifier alone with fewer steps or longer plans.

3.3 Closed loop

Table 3 comes from the PushT simulator, with the same 50 starting positions for every strategy. I stopped the run after 11 of its 14 strategies, so the gates at the looser threshold (validation quantile 0.5) are missing. The script only saves per-rollout records at the end, so these numbers come from its progress log: success counts with Wilson intervals and mean best coverage. There are no paired tests.

strategy	successes of 50	success [95% CI]	mean best coverage
DDPM-20 (reference)	24	0.48 [0.35, 0.61]	0.900
DDPM-10	26	0.52 [0.39, 0.65]	0.902
DDPM-5	25	0.50 [0.37, 0.63]	0.879
DDIM-10	31	0.62 [0.48, 0.74]	0.901
DDIM-5	32	0.64 [0.50, 0.76]	0.874
DDPM-20, chunk 12	27	0.54 [0.40, 0.67]	0.906
DDPM-20, chunk 15	32	0.64 [0.50, 0.76]	0.904
draft only	0	0.00 [0.00, 0.07]	0.086

strategy	successes of 50	success [95% CI]	mean best coverage
notebook rule, validation $q=0.1$	11	0.22 [0.13, 0.35]	0.802
previous agreement, validation $q=0.1$	27	0.54 [0.40, 0.67]	0.879
draft ensemble, validation $q=0.1$	17	0.34 [0.22, 0.48]	0.544

Table 3. Closed loop in gym-pusht, up to 300 steps per rollout, seeds 200000 to 200049. Success means block coverage above 95%. Gate thresholds are the 0.1 quantiles of each gate's validation signal.

Fewer steps don't visibly hurt the task. Every verifier sampler lands between 48% and 64% success, and all of their intervals overlap. DDIM and the 15-action plans come out a little ahead of the reference, but with 50 rollouts the intervals are about 13 points wide on each side, so differences that size can't be told apart. The closed loop doesn't show the open-loop penalty for 15-action plans either. For scale, the checkpoint's model card reports 65.4% success over 500 episodes.

The draft never succeeds. The notebook rule, which still pays for the verifier at every boundary, drops success from 48% to 22%. The ensemble gate drops it to 34%, and its mean coverage falls to 0.54, because whenever it trusts the draft the draft pushes the block the wrong way. The previous-agreement gate holds 54% at this threshold, where it skips only about 7% of verifier calls.

4. What carries over from LLM serving

Speculative decoding in LLMs checks several drafted tokens in one pass of the large model, and rejection sampling keeps the output distribution exactly the large model's (Leviathan et al. 2023, Chen et al. 2023). The first version's test needed the large model's answer, so it could never skip a call, and it had no exactness guarantee. An exact test can't run on whole actions here, because it needs each action's probability under both models, and a diffusion policy doesn't give that cheaply.

The version that does carry over works inside the denoising chain. De Bortoli et al. (2025) draft several denoising steps cheaply, check them in one batched call, and keep the model's output distribution exact. On their own PushT diffusion policy that cut model calls from 100 to about 27, and from 20 to about 9, with reward unchanged. The open question for this checkpoint is whether that beats plain DDIM-5 at equal compute. Closer to the first version, Spec-VLA (Wang et al. 2025) accepts drafted action tokens by distance, and TS-DP (Li et al. 2025) trains a transformer draft for a diffusion policy. Real-time chunking (Black et al. 2025) handles the latency problem directly, by planning the next chunk while the current one runs.

5. Limitations

- **The verifier saw the test episodes in training.** Only the draft is held out in the open-loop results. The closed-loop runs don't compare against demonstrations.
- **The reference isn't the checkpoint's own setting.** DDPM-20 matches the first version. The checkpoint defaults to 100 DDPM steps, and every speedup here is relative to a sampler already five times cheaper than that.

- **Step spacing.** All samplers use diffusers' default "leading" timestep spacing. With 5 steps, sampling starts at step 80 of 99, where the model expects 29% of the signal to remain, but it gets pure noise (Lin et al. 2024). "Trailing" spacing is the next check, and it could move the few-step rows either way.
- **Other work finds a closed-loop cost to fewer steps.** Prasad et al. (2024) report PushT coverage falling from 0.87 with 100 DDPM steps to 0.78 with 15 DDIM steps on their own checkpoint. Distilled few-step policies (Prasad et al. 2024, Wang et al. 2025) are the baseline this paper doesn't run.
- **Real time is checked, not simulated.** The per-frame times are averages. A loop that waits for each call stalls for 0.3 to 1.1 s at every boundary. Planning ahead from an older observation would fix that for the chunk-8 few-step samplers and costs about a third more error (section 3.1), but neither evaluation serves plans from older observations.
- **Timing comes from one laptop GPU in one session.** The benchmark ran with the machine otherwise idle, where the slowest call was at most 21% above the median. An earlier session run while other work competed for the machine had slow calls several times the median. Real-time claims need tail latencies measured under load, on the target hardware.
- **Open-loop draft plans use recorded states.** When the draft serves a plan, it sees the demonstrator's next states, which it wouldn't in a real rollout. That flatters the gates.
- **The splits are contiguous blocks.** Validation episodes turned out harder than test ones (reference error 75 vs 54), so the 5% budget meant 3.7 on validation and 2.7 on test.
- **The intervals cover episode-to-episode variation only.** They're conditional on one draft training run, one checkpoint and one block of test episodes.
- **One task, one checkpoint.** Nothing here tests another robot or policy.
- **The draft only sees the pusher position.** The block's pose is only in the image. A draft that sees the image, or one distilled from the verifier, would agree with it more often, and the gates would need that to compete with a 5-step sampler.
- **The closed-loop run is small and partial.** 50 rollouts leave intervals about 13 points wide on each side, so only large differences show. I stopped it before the gates at the looser threshold finished, and it hadn't saved per-rollout records, so there are no paired tests. Each boundary used one diffusion sample.

6. Reproducing

The open-loop numbers and figures come from `results/v2_openloop.json`, and the closed-loop numbers from `results/v2_closed_partial.json`. The numbers about the first version in section 1 come from its own writeup and notebook, which stay in the repo history.

```
uv sync --python 3.12 --extra dev --extra figures
uv run pytest -q
uv run spec_eval.py --out results/repro_openloop.json
uv run closed_loop.py --from results/repro_openloop.json --out results/repro_closed.json
uv run make_figures.py --open results/repro_openloop.json --closed results/repro_closed.json
```

The README lists the system packages Linux needs.

`spec_eval.py` also writes a local cache of every verifier plan, and `spec_eval.py --from-cache` recomputes every table and curve from it without running diffusion. The results files record package versions, script hashes and the checkpoint revision, and `results/environment.txt` lists every package from the run. Diffusion noise is seeded per episode and boundary. On this machine, with the same settings, the error and call counts repeat exactly. The DDPM step noise comes from the device's random generator, so other hardware draws different noise and won't match exactly, and timing never repeats exactly.

The original hackathon notebook stays in the repo as submitted. The paper supersedes its speedup, 10 Hz and "first port" claims.

References

- Black, Galliker, Levine. Real-Time Execution of Action Chunking Flow Policies. NeurIPS 2025. arXiv:2506.07339
- Chen, Borgeaud, Irving, Lespiau, Sifre, Jumper. Accelerating Large Language Model Decoding with Speculative Sampling. 2023. arXiv:2302.01318
- Chi et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. RSS 2023. arXiv:2303.04137
- De Bortoli, Galashov, Gretton, Doucet. Accelerated Diffusion Models via Speculative Sampling. ICML 2025. arXiv:2501.05370
- Ho, Jain, Abbeel. Denoising Diffusion Probabilistic Models. NeurIPS 2020. arXiv:2006.11239
- Kim et al. Speculative Decoding with Big Little Decoder. NeurIPS 2023. arXiv:2302.07863
- Leviathan, Kalman, Matias. Fast Inference from Transformers via Speculative Decoding. ICML 2023. arXiv:2211.17192
- Li et al. TS-DP: Reinforcement Speculative Decoding For Temporal Adaptive Diffusion Policy Acceleration. 2025. arXiv:2512.15773
- Lin, Liu, Li, Yang. Common Diffusion Noise Schedules and Sample Steps are Flawed. WACV 2024. arXiv:2305.08891
- Prasad et al. Consistency Policy: Accelerated Visuomotor Policies via Consistency Distillation. RSS 2024. arXiv:2405.07503
- Song, Meng, Ermon. Denoising Diffusion Implicit Models. ICLR 2021. arXiv:2010.02502

- Wang et al. One-Step Diffusion Policy: Fast Visuomotor Policies via Diffusion Distillation. ICML 2025. arXiv:2410.21257
- Wang et al. Spec-VLA: Speculative Decoding for Vision-Language-Action Models with Relaxed Acceptance. EMNLP 2025. arXiv:2507.22424